



Name: K. Archana

Course : DAA

Reg No: 192524209

Code: CSA0618



Experiment 1: Linear Search (Iterative)

```
a = [10, 25, 30, 45, 50]
```

```
key = 30
```

```
result = linear_search(a, key)
```

```
if result != -1:
```

```
    print("Key found at index", result)
```

```
else:
```

```
    print("Key not found")
```

- Sample Input: [10, 25, 30, 45, 50], Key = 30

- Sample Output: Key found at index 2

The screenshot shows a Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 arr = [10, 25, 30, 45, 50]
2 key = 30
3
4 found = False
5
6 for i in range(len(arr)):
7     if arr[i] == key:
8         print("Key found at index", i)
9         found = True
10        break
11
12 if not found:
13     print("Key not found")
```

The output panel on the right displays the result: "Key found at index 2".

Experiment 2: Binary Search (Recursive)

```
def binary_search(a, low, high, key):
```

```
    if low > high:
```

```
        return -1
```

```
    mid = (low + high) // 2
```

```

if a[mid] == key:
    return mid
elif key < a[mid]:
    return binary_search(a, low, mid - 1, key)
else:
    return binary_search(a, mid + 1, high, key)

```

```

a = [5, 10, 15, 20, 25]
key = 20

```

```

result = binary_search(a, 0, len(a) - 1, key)

```

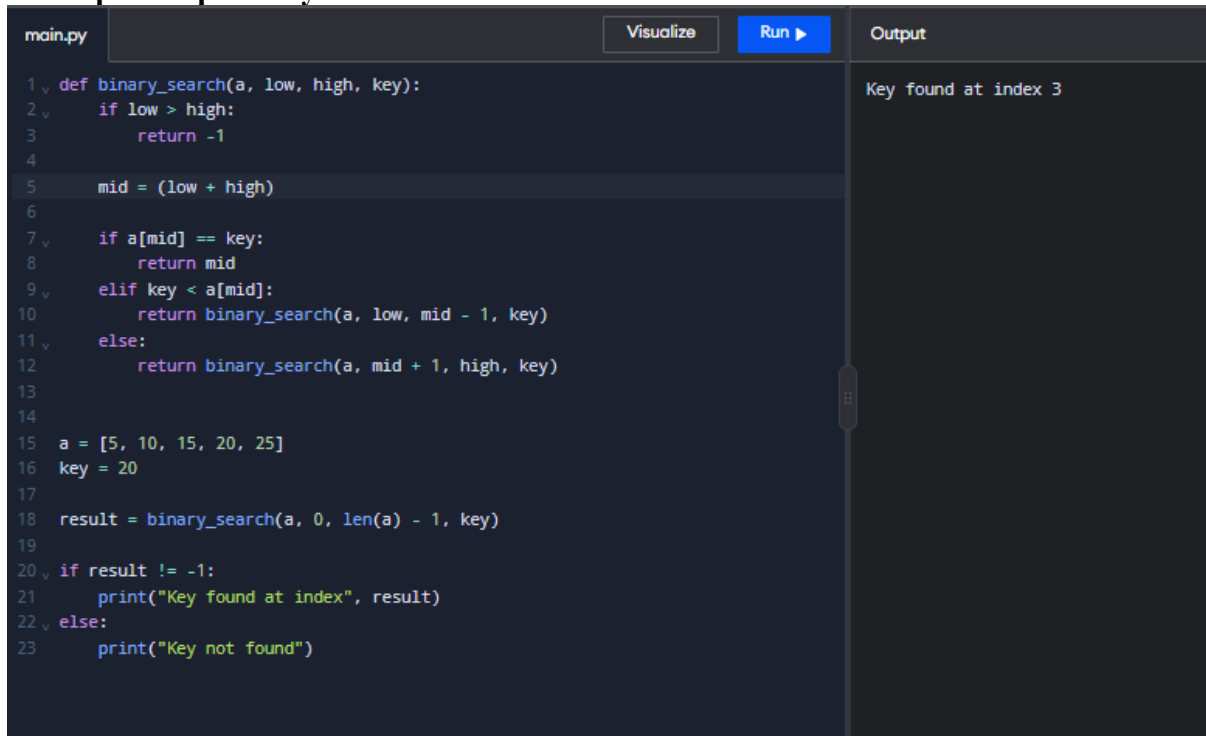
```

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")

```

• **Sample Input:** [5, 10, 15, 20, 25], Key = 20

• **Sample Output:** Key found at index 3



```

main.py Visualize Run ▶ Output
1 def binary_search(a, low, high, key):
2     if low > high:
3         return -1
4
5     mid = (low + high)
6
7     if a[mid] == key:
8         return mid
9     elif key < a[mid]:
10        return binary_search(a, low, mid - 1, key)
11    else:
12        return binary_search(a, mid + 1, high, key)
13
14
15 a = [5, 10, 15, 20, 25]
16 key = 20
17
18 result = binary_search(a, 0, len(a) - 1, key)
19
20 if result != -1:
21     print("Key found at index", result)
22 else:
23     print("Key not found")

```

Key found at index 3

Experiment 3: Factorial (Iterative vs Recursive).

```

def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

```

```

def factorial_iterative(n):
    fact = 1

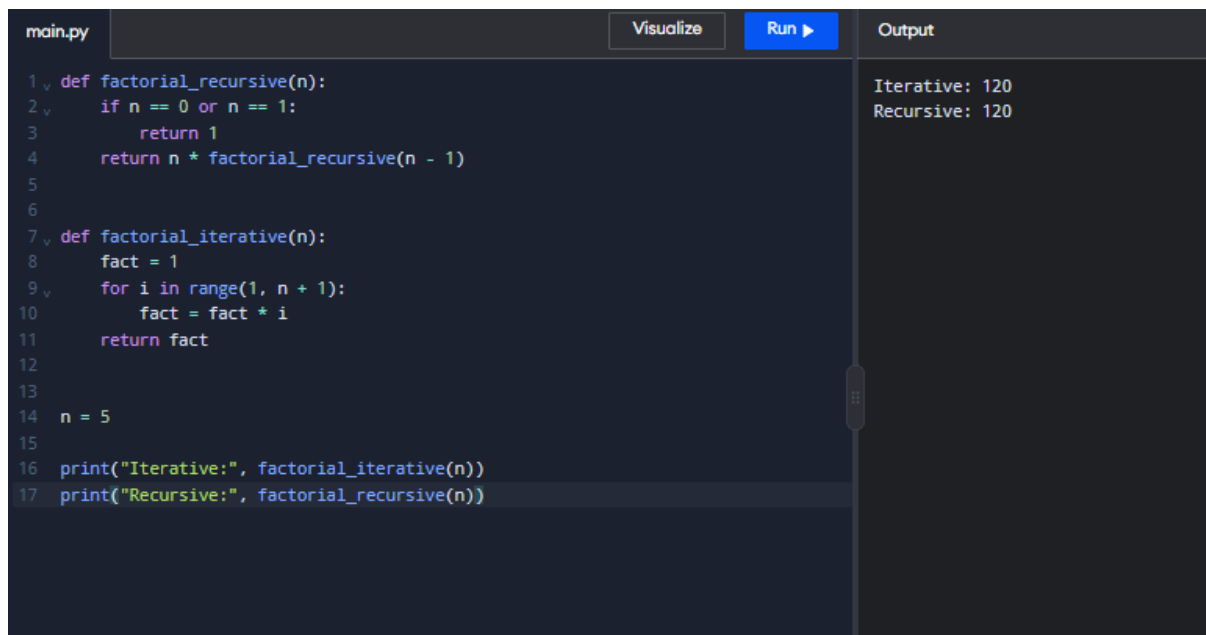
```

```
for i in range(1, n + 1):
    fact = fact * i
return fact
```

n = 5

```
print("Iterative:", factorial_iterative(n))
print("Recursive:", factorial_recursive(n))
```

- **Sample Input: n = 5**
- **Sample Output: 120**

A screenshot of a code editor interface. The editor has a dark theme. At the top, there's a tab labeled 'main.py'. To the right of the tab are two buttons: 'Visualize' and 'Run' (with a play icon). The main area of the editor contains Python code for calculating factorials. The code defines two functions: 'factorial_recursive' and 'factorial_iterative'. Below these functions, it sets 'n = 5' and prints the results of both functions. To the right of the code editor is a panel labeled 'Output'. It displays the results of the program execution: 'Iterative: 120' and 'Recursive: 120'.

```
main.py Visualize Run ▶ Output
1 def factorial_recursive(n):
2     if n == 0 or n == 1:
3         return 1
4     return n * factorial_recursive(n - 1)
5
6
7 def factorial_iterative(n):
8     fact = 1
9     for i in range(1, n + 1):
10         fact = fact * i
11     return fact
12
13
14 n = 5
15
16 print("Iterative:", factorial_iterative(n))
17 print("Recursive:", factorial_recursive(n))

Output
Iterative: 120
Recursive: 120
```

Experiment 4: Fibonacci Series

```
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

```
def fibonacci_iterative(n):
    a, b = 0, 1
    result = []

    for i in range(n):
        result.append(a)
        a, b = b, a + b

    return result
```

```
n = 6
```

```
print("Iterative:", fibonacci_iterative(n))
```

```
recursive_result = []
```

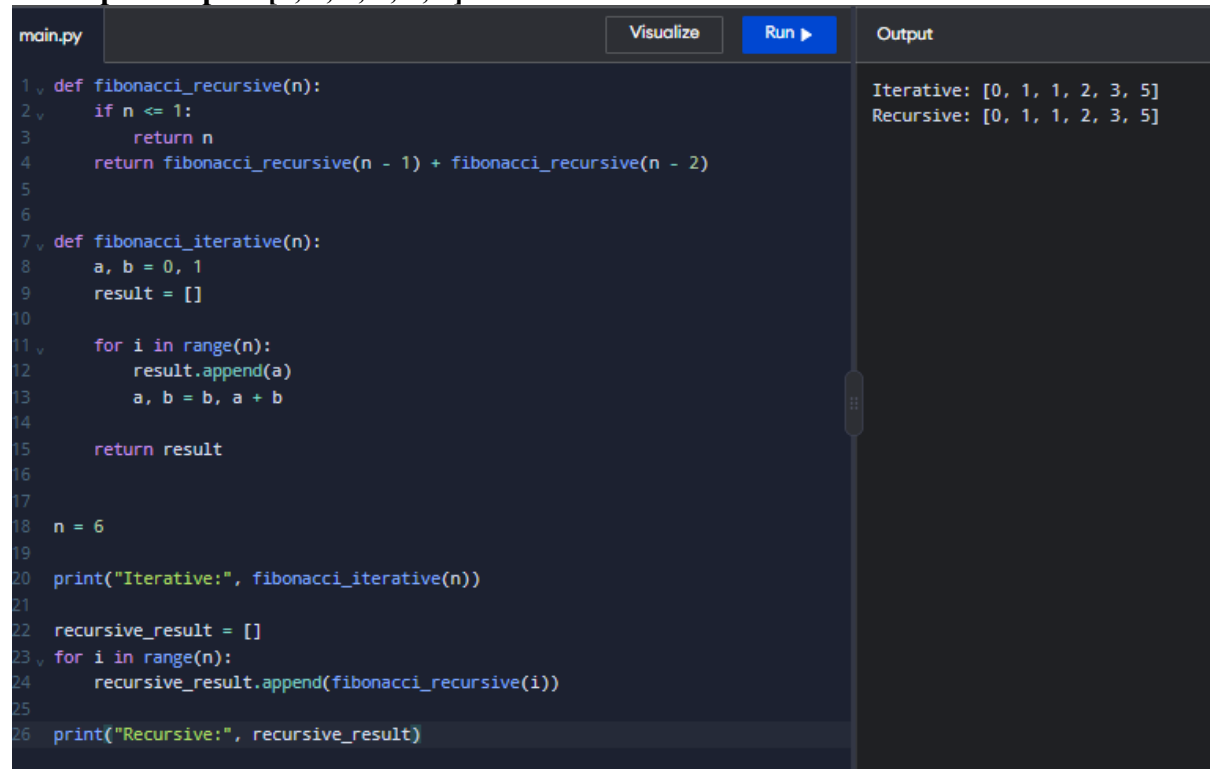
```
for i in range(n):
```

```
    recursive_result.append(fibonacci_recursive(i))
```

```
print("Recursive:", recursive_result)
```

- Sample Input: n = 6

- Sample Output: [0, 1, 1, 2, 3, 5]



```
main.py Visualize Run ▶ Output
1 def fibonacci_recursive(n):
2     if n <= 1:
3         return n
4     return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
5
6
7 def fibonacci_iterative(n):
8     a, b = 0, 1
9     result = []
10
11     for i in range(n):
12         result.append(a)
13         a, b = b, a + b
14
15     return result
16
17
18 n = 6
19
20 print("Iterative:", fibonacci_iterative(n))
21
22 recursive_result = []
23 for i in range(n):
24     recursive_result.append(fibonacci_recursive(i))
25
26 print("Recursive:", recursive_result)
```

Iterative: [0, 1, 1, 2, 3, 5]
Recursive: [0, 1, 1, 2, 3, 5]

Experiment 5: Matrix Multiplication

```
A = [[1, 2],
      [3, 4]]
```

```
B = [[5, 6],
      [7, 8]]
```

```
result = [[0, 0],
           [0, 0]]
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        for k in range(2):
```

```
            result[i][j] += A[i][k] * B[k][j]
```

```
print(result)
```

- Sample Input: A=[[1,2],[3,4]], B=[[5,6],[7,8]]

- Sample Output: [[19,22],[43,50]]

```
main.py Visualize Run ▶ Output
1 ✓ A = [[1, 2],
2       [3, 4]]
3
4 ✓ B = [[5, 6],
5       [7, 8]]
6
7 ✓ result = [[0, 0],
8             [0, 0]]
9
10 ✓ for i in range(2):
11     for j in range(2):
12         for k in range(2):
13             result[i][j] += A[i][k] * B[k][j]
14
15 print(result)
```

[[19, 22], [43, 50]]

Experiment 6: Merge Sort

```
def merge_sort(a):
```

```
    if len(a) <= 1:
```

```
        return a
```

```
    mid = len(a) // 2
```

```
    left = merge_sort(a[:mid])
```

```
    right = merge_sort(a[mid:])
```

```
    result = []
```

```
    i = 0
```

```
    j = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] < right[j]:
```

```
            result.append(left[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right[j])
```

```
            j += 1
```

```
    result.extend(left[i:])
```

```
    result.extend(right[j:])
```

```
    return result
```

```
a = [38, 27, 43, 3, 9, 82, 10]
```

```
print(merge_sort(a))
```

• **Sample Input:** [38, 27, 43, 3, 9, 82, 10]

• **Sample Output:** [3, 9, 10, 27, 38, 43, 82]

```
main.py Visualize Run ▶ Output
1 def merge_sort(a):
2     if len(a) <= 1:
3         return a
4
5     mid = len(a) // 2
6
7     left = merge_sort(a[:mid])
8     right = merge_sort(a[mid:])
9
10    result = []
11    i = 0
12    j = 0
13
14    while i < len(left) and j < len(right):
15        if left[i] < right[j]:
16            result.append(left[i])
17            i += 1
18        else:
19            result.append(right[j])
20            j += 1
21
22    result.extend(left[i:])
23    result.extend(right[j:])
24
25    return result
26
27
28 a = [38, 27, 43, 3, 9, 82, 10]
```

[3, 9, 10, 27, 38, 43, 82]

Experiment 7: Quick Sort

def quick_sort(a):

if len(a) <= 1:

return a

pivot = a[0]

left = [x for x in a[1:] if x <= pivot]

right = [x for x in a[1:] if x > pivot]

return quick_sort(left) + [pivot] + quick_sort(right)

a = [10, 7, 8, 9, 1, 5]

print(quick_sort(a))

• **Sample Input:** [10, 7, 8, 9, 1, 5]

• **Sample Output:** [1, 5, 7, 8, 9, 10]

main.py	Visualize	Run ▶	Output
<pre>1 def quick_sort(a): 2 if len(a) <= 1: 3 return a 4 5 pivot = a[0] 6 7 left = [x for x in a[1:] if x <= pivot] 8 right = [x for x in a[1:] if x > pivot] 9 10 return quick_sort(left) + [pivot] + quick_sort(right) 11 12 13 a = [10, 7, 8, 9, 1, 5] 14 15 print(quick_sort(a))</pre>			<pre>[1, 5, 7, 8, 9, 10]</pre>

Experiment 8: Tower of Hanoi

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)

    print("Move disk", n, "from", source, "to", destination)

    tower_of_hanoi(n - 1, auxiliary, source, destination)
```

n = 3

tower_of_hanoi(n, 'A', 'B', 'C')

● Sample Input: n = 3

● Sample Output: Sequence of moves.

main.py	Visualize	Run ▶	Output
<pre>1 def tower_of_hanoi(n, source, auxiliary, destination): 2 if n == 1: 3 print("Move disk 1 from", source, "to", destination) 4 return 5 6 tower_of_hanoi(n - 1, source, destination, auxiliary) 7 8 print("Move disk", n, "from", source, "to", destination) 9 10 tower_of_hanoi(n - 1, auxiliary, source, destination) 11 12 13 n = 3 14 15 tower_of_hanoi(n, 'A', 'B', 'C')</pre>			<pre>Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C</pre>

Experiment 9: GCD (Euclidean Algorithm)

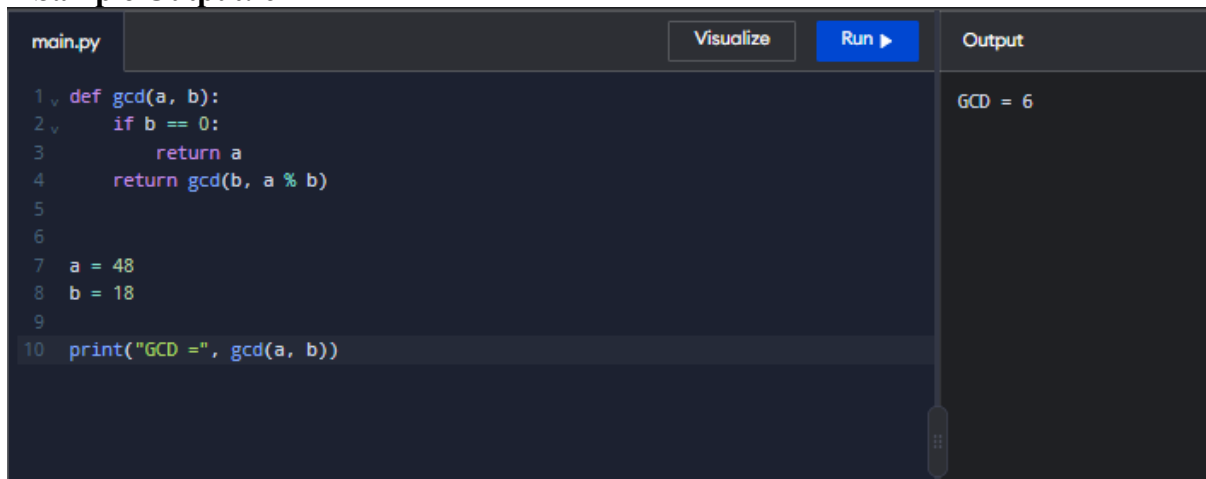
```
def gcd(a, b):
    if b == 0:
        return a
```

```
return gcd(b, a % b)
```

```
a = 48  
b = 18
```

```
print("GCD =", gcd(a, b))
```

- **Sample Input:** a=48, b=18
- **Sample Output:** 6



```
main.py Visualize Run ▶ Output  
1 def gcd(a, b):  
2     if b == 0:  
3         return a  
4     return gcd(b, a % b)  
5  
6  
7 a = 48  
8 b = 18  
9  
10 print("GCD =", gcd(a, b))  
  
GCD = 6
```

Experiment 10: Insertion Sort

```
def insertion_sort(a):
```

```
    for i in range(1, len(a)):  
        key = a[i]  
        j = i - 1
```

```
        while j >= 0 and a[j] > key:  
            a[j + 1] = a[j]  
            j -= 1
```

```
        a[j + 1] = key
```

```
a = [12, 11, 13, 5, 6]
```

```
insertion_sort(a)
```

```
print(a)
```

- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]

```
main.py Visualize Run ▶ Output
1 def insertion_sort(a):
2     for i in range(1, len(a)):
3         key = a[i]
4         j = i - 1
5
6         while j >= 0 and a[j] > key:
7             a[j + 1] = a[j]
8             j -= 1
9
10        a[j + 1] = key
11
12
13 a = [12, 11, 13, 5, 6]
14
15 insertion_sort(a)
16
17 print(a)
```

[5, 6, 11, 12, 13]

Experiment 11: Heap Sort

- Sample Input: [4, 10, 3, 5, 1]
- Sample Output: [1, 3, 4, 5, 10]

```
def heapify(a, n, i):
```

```
    largest = i
```

```
    left = 2 * i + 1
```

```
    right = 2 * i + 2
```

```
    if left < n and a[left] > a[largest]:
        largest = left
```

```
    if right < n and a[right] > a[largest]:
        largest = right
```

```
    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)
```

```
def heap_sort(a):
```

```
    n = len(a)
```

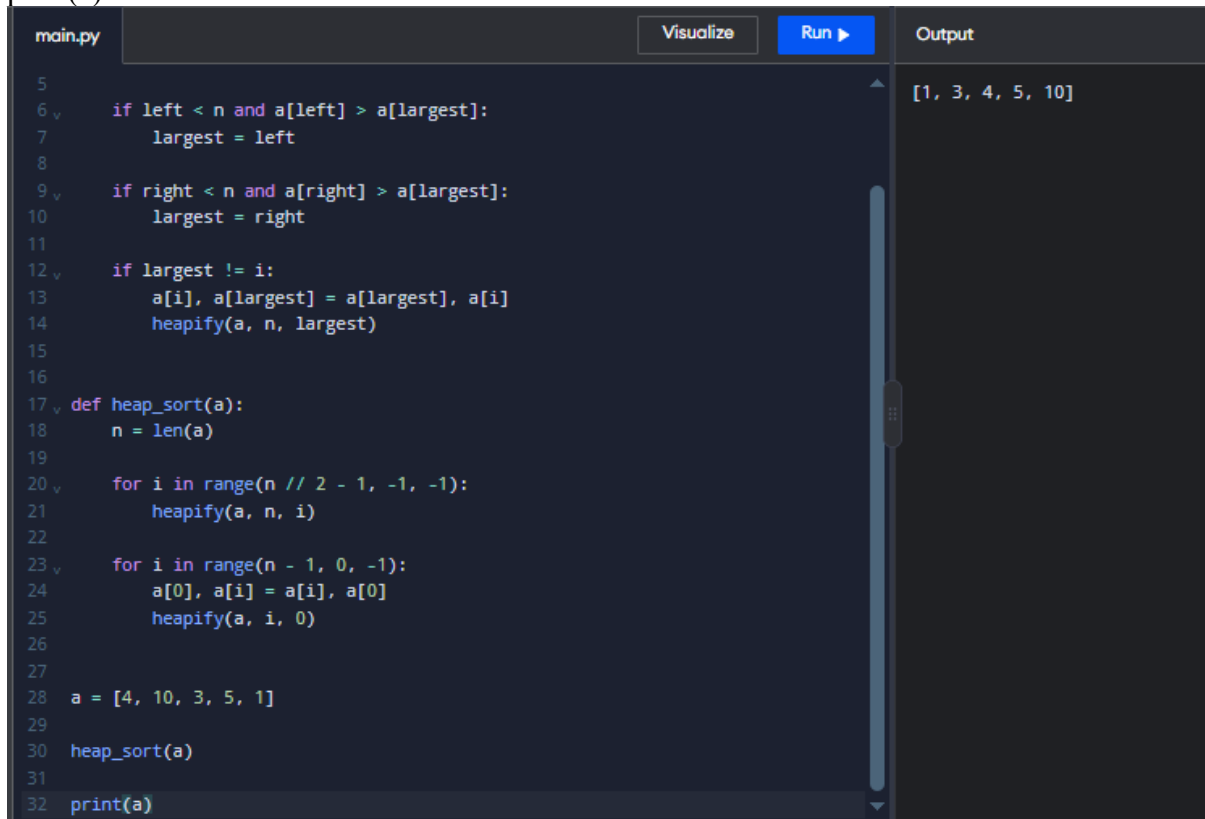
```
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
```

```
    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)
```

```
a = [4, 10, 3, 5, 1]
```

```
heap_sort(a)
```

print(a)



```
main.py Visualize Run ▶ Output
5
6 ✓ if left < n and a[left] > a[largest]:
7     largest = left
8
9 ✓ if right < n and a[right] > a[largest]:
10     largest = right
11
12 ✓ if largest != i:
13     a[i], a[largest] = a[largest], a[i]
14     heapify(a, n, largest)
15
16
17 ✓ def heap_sort(a):
18     n = len(a)
19
20 ✓ for i in range(n // 2 - 1, -1, -1):
21     heapify(a, n, i)
22
23 ✓ for i in range(n - 1, 0, -1):
24     a[0], a[i] = a[i], a[0]
25     heapify(a, i, 0)
26
27
28 a = [4, 10, 3, 5, 1]
29
30 heap_sort(a)
31
32 print(a)
```

[1, 3, 4, 5, 10]

Experiment 12: Counting Inversions

```
def count_inversions(a):
```

```
    count = 0
```

```
    for i in range(len(a)):
```

```
        for j in range(i + 1, len(a)):
```

```
            if a[i] > a[j]:
```

```
                count += 1
```

```
    return count
```

```
a = [2, 4, 1, 3, 5]
```

```
print("Number of inversions:", count_inversions(a))
```

- **Sample Input:** [2, 4, 1, 3, 5]

- **Sample Output:** 3

```
main.py Visualize Run ▶ Output
1 def count_inversions(a):
2     count = 0
3
4     for i in range(len(a)):
5         for j in range(i + 1, len(a)):
6             if a[i] > a[j]:
7                 count += 1
8
9     return count
10
11
12 a = [2, 4, 1, 3, 5]
13
14 print("Number of inversions:", count_inversions(a))
```

Number of inversions: 3

Experiment 13: Maximum Subarray Sum

- Sample Input: [-2, -3, 4, -1, -2, 1, 5, -3]
- Sample Output: 7

```
main.py Visualize Run ▶ Output
1 def max_subarray_sum(a):
2     current = a[0]
3     maximum = a[0]
4
5     for i in range(1, len(a)):
6         current = max(a[i], current + a[i])
7         maximum = max(maximum, current)
8
9     return maximum
10
11
12 a = [-2, -3, 4, -1, -2, 1, 5, -3]
13
14 print("Maximum subarray sum:", max_subarray_sum(a))
```

Maximum subarray sum: 7

Experiment 14: Exponentiation

def power(x, n):

if n == 0:
 return 1

half = power(x, n // 2)

if n % 2 == 0:
 return half * half
 else:
 return x * half * half

x = 2
n = 10

print("Result:", power(x, n))

- Sample Input: x=2, n=10

- **Sample Output: 1024**



```
main.py Visualize Run ▶ Output
1 def power(x, n):
2     if n == 0:
3         return 1
4
5     half = power(x, n // 2)
6
7     if n % 2 == 0:
8         return half * half
9     else:
10        return x * half * half
11
12
13 x = 2
14 n = 10
15
16 print("Result:", power(x, n))

Result: 1024
```

Experiment 15: Closest Pair of Points

```
import math
```

```
def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0]) ** 2 +
                     (p1[1] - p2[1]) ** 2)
```

```
def closest_pair(points):
    min_distance = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])

            if d < min_distance:
                min_distance = d
                pair = (points[i], points[j])

    return pair, min_distance
```

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

```
pair, d = closest_pair(points)
```

```
print("Closest pair:", pair)
```

```
print("Distance:", d)
```

- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]

- **Sample Output:** Closest pair (1,2)-(3,1), Distance = $\sqrt{5}$

main.py	Visualize	Run ▶	Output
<pre>1 import math 2 3 def distance(p1, p2): 4 return math.sqrt((p1[0] - p2[0])**2 + 5 (p1[1] - p2[1])**2) 6 7 8 def closest_pair(points): 9 min_distance = float('inf') 10 pair = None 11 12 for i in range(len(points)): 13 for j in range(i + 1, len(points)): 14 d = distance(points[i], points[j]) 15 16 if d < min_distance: 17 min_distance = d 18 pair = (points[i], points[j]) 19 20 return pair, min_distance 21 22 23 points = [(1, 2), (4, 5), (7, 8), (3, 1)] 24 25 pair, d = closest_pair(points) 26 27 print("Closest pair:", pair) 28 print("Distance:", d)</pre>			<pre>Closest pair: ((1, 2), (3, 1)) Distance: 2.23606797749979</pre>